The University of the West Indies at Mona
Department of Computing
COMP 3101 – Operating Systems
Semester I, 2026/2027
Assignment 2 – 10%
Date Due: October 27th, 2026 @ 11:59 pm

Part A – 15 marks

1. Begin by writing a recursive C function that calculates the nth fibonacci number, this will be used in the main assignment problem. The fibonacci numbers are a sequence given by:

$$f_1 = 1, f_2 = 1, f_n = f_{n-1} + f_{n-2}, \text{ for } n > 3$$

Write a C program that generates a timer profile of the current process. Your time measurements should have seconds, and microseconds accuracy. It should be able to display four (4) different timer measurements for the current process:

- Wall Time Spent Executing - The amount of time the process spent from start to finish. (i.e. the Real Time)
- Total Time Spent on CPU - The amount of time the process spent executing instructions on the CPU plus the amount of time that the kernel was doing work on behalf of the process. (i.e. the Profile Time)
- Time Spent in User Space - The amount of time the process spent executing instructions on the CPU. (i.e. the Virtual Time)
- Time Spent in Kernel Space - The amount of time the kernel spent doing work on behalf of the process.

For each interval that you wish to record, set up an interval timer that raises a signal every one (1) second. Each second, when the signal is raised, some counter should be used to keep track of the number of seconds that have elapsed. In the end, the counter together with the present contents of the given interval timer will be sufficient to obtain the number of seconds and microseconds that have elapsed since the process began.

Write the code for profiling the process in a general manner so that it can be re-used for multiple processes. This will be necessary for the next part of the assignment given below.

Next adjust your program so that it spawns new processes. The main program (the parent process) should spawn a first child that is given to compute the nth fibonacci number, where n is the second command line parameter (i.e. argv[2]). Immediately after spawning the first child, the parent should spawn a second child that is given to compute the mth fibonacci number, where m is the third command line parameter (i.e. argv[3]). After spawning the second child, the parent itself should then proceed to compute the fibonacci number for the first command line parameter (i.e. argv[1]). After the parent has finished computing, it should wait on the first and second child processes to complete before continuing.

After each child has finished computing its assigned fibonacci number, it's timer profile should be printed. The parent's profile should be printed after it has done waiting on the children.

Your program should be called Fibonacci_timers.c. Following is an example of how such a program should work.

```
$ ./fibonacci_timers 43 44 45
[Child 1] fibonacci(44) = 701408733
Wall Time Spent Executing : 42 seconds, 440000 microseconds
Total Time Spent on CPU : 15 seconds, 290000 microseconds
Time Spent in User-Space : 15 seconds, 280000 microseconds
Time Spent in Kernel-Space: 0 seconds, 10000 microseconds

[Child 2] fibonacci(45) = 1134903170
Wall Time Spent Executing : 52 seconds, 310000 microseconds
Total Time Spent on CPU : 24 seconds, 310000 microseconds
Time Spent in User-Space : 24 seconds, 220000 microseconds
Time Spent in Kernel-Space: 0 seconds, 90000 microseconds

[Parent] fibonacci(43) = 433494437
Wall Time Spent Executing : 52 seconds, 1000000 microseconds
Total Time Spent on CPU : 9 seconds, 1000000 microseconds
Time Spent in User-Space : 9 seconds, 1000000 microseconds
Time Spent in Kernel-Space: 0 seconds, 0 microseconds
$
```

[15 marks]

Part B – 15 marks

Suppose there are five memory blocks of sizes 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB, in that order. A set of processes requiring 212 KB, 417 KB, 112 KB, and 426 KB (in that order) need to be allocated memory. How would the memory be allocated for these processes using the following algorithms: first-fit, best-fit, worst-fit, and next-fit? Which algorithm makes the most efficient use of memory?

[15 marks]

Submission Instructions:

Zip your files into a single zip folder containing all programming files (Question 1 in Part A) and one single text file containing the answers to all the questions with discussion in Part B (Name the text file "ID#Number_A2PartB.docx", e.g. "808000326_A2PartB.docx"). Also please include your signed UWI Plagiarism Declaration form in your zip folder as well. Name the zip folder "ID#Number_A2.zip", e.g. "808000326_A2.zip". Please upload the zip folder to the submission link on myElearning. Only 1 submission is allowed, so be careful that your file submitted is the final version.